The *cgi* module does not differentiate between *get* and *post* actions. The *getvalue* method picks up the value from the form. The second parameter passed to this method assigns a default value in case the value is not available. The method *process_form* currently does nothing. It could contain the code for storing the data in a database, or whatever else you may need.

Adding some style

However, the impression may be that pages generated through Python are plain and ugly. We can make them even uglier, but colourful! The look and feel will be controlled by a style sheet exactly like the usual HTML pages. So, replace the <head></head> line above by:

```
<head>
<style type="text/css">
p {margin-left: 50px}
body {background-color: tan}
input {background-color: yellow}
button {background-color: orange}
</style>
</head>
```

The page now will be more colourful. Incidentally, the styles can usually be read from a CSS file.

WSGI

Now that you know how to write a simple application for the Web, you may want to write an application server in Python. Chances are you would use one like Django, Turbo-Gears, Pylons, etc. But you can use the Web server gateway interface as well, which is supported by various application servers. You can find out more about it at *www.wsgi.org* or just by searching for 'wsgi tutorials', e.g., http://webpython.codepoint.net/wsgi_tutorial. Try the following simple code in a file *test_app.py*:

```
#!/usr/bin/python
from wsgiref.simple_server import make_server
def application(environ, start_response):
  keys_of_interest = ['PATH_INFO', 'QUERY_STRING','REQUEST_METHOD']
  env_vars = (key + ':' + value for key, value in environ.items()
              if key in keys_of_interest)
  msg = '\n'.join(env_vars)
  status = '200 OK'
  response_headers = [('Content-Type', 'text/plain')]
  start_response(status, response_headers)
  return ["Environment Variables of Interest\n\n", msg]
# Start the server and handle just one request
httpd = make_server('localhost', 8080, application)
httpd.handle_request()
```

To create a Web server, you will need to have a method called *application* (not mandatory, but preferable). The method requires two parameters, *environ* and *start_response*, and returns a list of strings to be displayed. The first parameter is a dictionary containing various environment variables. Your program displays a few of them related to the Web page. The second parameter is a method you call to, well, start the response for the request.

You can convert your code into a Web server using the simple reference server included in Python. Your server would normally keep handling requests and instead of calling *handle_request*, you would call *serve_forever*.

You may notice the use of *()* instead of *[]* to define *env_vars*, using list comprehensions. The use of square brackets creates a list while the use of parentheses creates a generator for the list. Using a generator saves the creation of a temporary list, which can be especially beneficial when dealing with potentially large lists.

You would run the application from the command prompt, *python test_app.py*, and by pointing the browser to *localhost:8080/anything?anyvar=anyvalue*. The result would be as follows:

```
Environment Variables of Interest

REQUEST_METHOD:GET
PATH_INFO:/anything
QUERY_STRING:anyvar=anyval
```

You can change the CGI application you wrote to work with WSGI instead. Your file, *application.py*, should then include the following:

```
#!/usr/bin/python
from wsgiref.simple_server import make_server
from cgi import parse_qs
def application(environ, start_response):
  path_info = environ['PATH_INFO']
  if path_info.endswith('form.py'):
    # for post method, params in a file object
    req_size = int(environ['CONTENT_LENGTH'])
    params = parse_qs(
      environ['wsgi.input'].read(req_size))
    response = process_form(params)
  else:
    response = display_form()
  status = '200 OK'
  response_headers = [('Content-Type', 'text/html')]
  start_response('200 OK', [('Content-Type', 'text/html')])
  return [response]
httpd = make_server('localhost', 8080, application)
httpd.serve_forever()
```

The core application code is not very different from the earlier one. The form responds with a *POST* method and calls *form.py* because the form had called it in the